TEKsystems Global Services | WORKFORCE DEVELOPMENT

QUALITY ASSURANCE INTRODUCTION



QUALITY ASSURANCE INTRODUCTION

A three-day, hands-on course on the foundations of software quality: planning, designing, automating, and reporting on tests the way modern teams do.

- Half lecture, half labs
- One sample application, BookNook, used all week
- Python and JavaScript, Cypress, Postman, and GitHub Actions



© 2026 by Innovation In Software Corporation

LOGISTICS



Class Hours:

- Class will be 10 am–5 pm ET
- Lunch at 1 pm (75 minutes)
- Breaks at 11:30 am and 3 pm (20 minutes)



Devices:

- Turn off or set electronic devices to silent (not vibrate)
- Reading or attending to devices can be distracting to other students
- Try to delay until breaks or after class

© 2026 by Innovation In Software Corporation

INSTRUCTOR: ROD DAVISON

About

- 54 years experience in IT related areas
  - 40 years in AI related work
- QA and testing
  - 25 years industrial and engineering testing
  - 20 years market research, social sciences testing and experimental design
- Consulted to
  - Major US and Canadian banks
  - US, Canadian and UK government agencies
  - Fortune 100 companies
  - Defense contractors



© 2026 by Innovation In Software Corporation

INTRODUCE YOURSELF

Time to introduce yourself:

- Name
- Your role in the organization
- Your QA/Testing experience
- Expectations for this training



© 2026 by Innovation In Software Corporation

THE COURSE AT A GLANCE

- **Day 1:** QA foundations, test planning, exploratory and scripted testing, and writing test cases
- **Day 2:** Test-driven development, automated testing, UI testing, and API testing
- **Day 3:** Triage and debugging, bug reports and metrics, CI pipelines, and QA antipatterns

Every lab builds on the one before. The test cases you write today become the automated tests you build tomorrow and the pipeline you run on Day 3.

© 2026 by Innovation In Software Corporation

DAY 1: FOUNDATIONS

- What quality assurance is and why it matters
- The QA engineer's role in the software development lifecycle
- Test planning and risk-based prioritization
- Exploratory vs. scripted testing
- Functional and non-functional testing
- Writing test cases from user stories

© 2026 by Innovation In Software Corporation

MEET BOOKNOOK

BookNook is the online bookstore you will test all week. Customers browse and search a catalog, add books to a cart, apply discount codes, and log in to their accounts.

BookNook has a requirements document, a REST API, and a web front end. It also has bugs, planted on purpose. Finding, reporting, and preventing them is your job for the next three days.



© 2026 by Innovation In Software Corporation

WHAT IS QUALITY?

Quality is how well software meets the needs of the people who use it. That includes doing the right thing (correct results), doing it reliably, and doing it in a way people can actually use.

Quality is never just "no bugs." A checkout page with no crashes that charges the wrong amount is not quality software. Neither is a perfectly correct page that takes 30 seconds to load or can't be used with a screen reader.

Every team defines quality through its requirements, and every requirement is a promise someone will check.



© 2026 by Innovation In Software Corporation

WHAT IS QUALITY ASSURANCE?

Quality assurance (QA) is the set of activities that make sure a team builds quality in, rather than inspecting it in at the end.

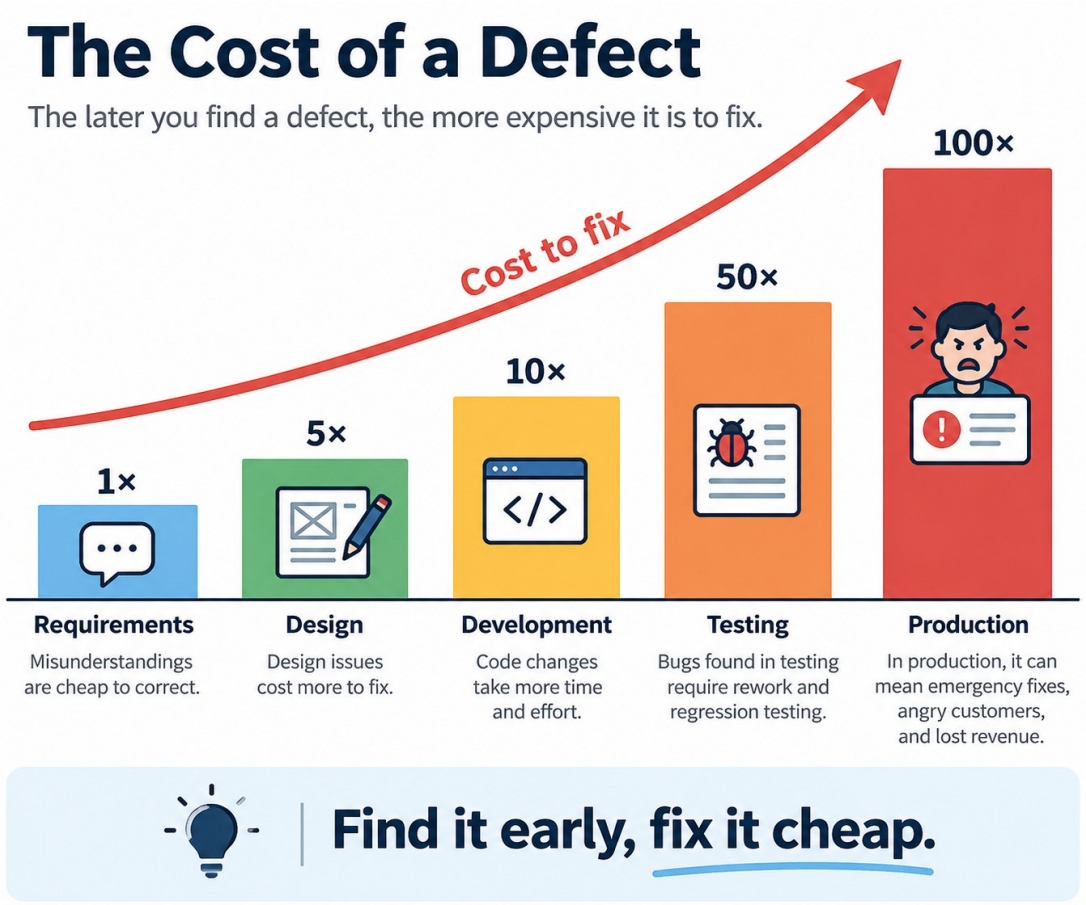
- QA is about the **process**: how requirements are written, reviewed, built, and checked
- Its goal is to **prevent** defects, not only to find them
- It runs through the whole lifecycle, from the first requirement to the release

© 2026 by Innovation In Software Corporation

WHY QA MATTERS: THE COST OF A DEFECT

A defect gets more expensive the later it is found. A vague requirement caught in review costs a conversation. The same mistake caught in production costs a hotfix, support tickets, refunds, and customer trust.

- Found in requirements: a question and a rewrite
- Found in development: a code change and a retest
- Found in testing: a bug report, a fix, a retest, and a delay
- Found in production: all of the above, plus unhappy customers



Illustrative Figures

© 2026 by Innovation In Software Corporation

THE COST OF POOR QUALITY

Picture launch week for an online electronics store. A typo in a price import lists a $499 laptop at $49, the word spreads on social media, and thousands of orders go out before anyone notices. The fix takes ten minutes. The refunds, support calls, and headlines take months.

Most quality failures aren't dramatic crashes. They're small rules nobody checked, in places where money, data, or trust is on the line.



© 2026 by Innovation In Software Corporation

SEVEN PRINCIPLES OF TESTING

These principles hold on every project, whatever the tools or methodology.

Principle	What it means for you
Testing shows bugs are **present**	It can never prove there are none
Exhaustive testing is **impossible**	So you prioritize by risk
Test **early**	Defects are cheapest to fix at the start
Defects **cluster**	A few areas hold most of the bugs; look harder there
Tests **wear out**	Running the same tests finds fewer new bugs over time; refresh them
Testing is **context dependent**	A bookstore and a pacemaker need different testing
"No bugs found" isn't success	Software can be bug-free and still not meet users' needs

© 2026 by Innovation In Software Corporation

QA VS. QC VS. TESTING

These three terms are often used as if they mean the same thing. They don't.

	Quality Assurance (QA)	Quality Control (QC)	Testing
Focus	The **process**	The **product**	Specific **checks**
Goal	Prevent defects	Find defects	Run a check, record a result
Example	Reviewing requirements for ambiguity	Inspecting a build before release	Running a login test case
When	Throughout the lifecycle	After something is built	During development and QC

© 2026 by Innovation In Software Corporation

THE QA ENGINEER IN THE SDLC

The software development lifecycle (SDLC) is the path from an idea to running software. A QA engineer adds value at every stage, not only at the end.

- **Requirements:** spot vague, missing, or conflicting rules before code exists
- **Design:** plan what to test, how, and in what order
- **Development:** write test cases and automated tests alongside the code
- **Testing:** run tests, explore, and report defects with evidence
- **Release and beyond:** decide readiness against exit criteria and learn from escaped bugs

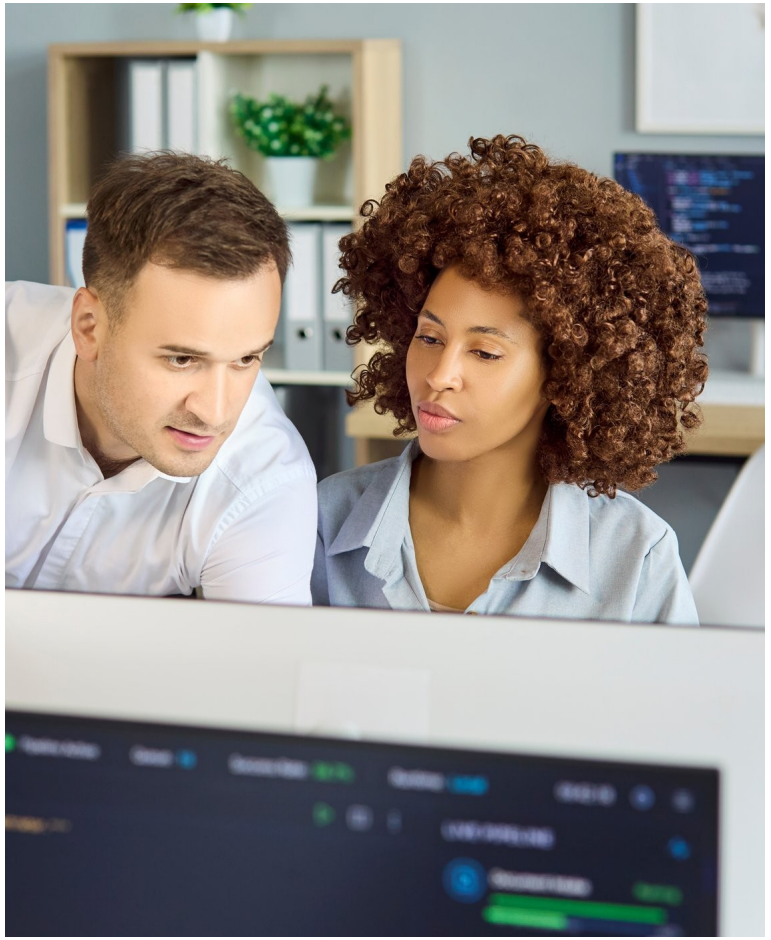


© 2026 by Innovation In Software Corporation

A DAY IN THE LIFE OF A QA ENGINEER

A QA engineer asks questions for a living. Is this requirement testable? What could break? What happens if a user does something unexpected? Is this release ready?

The role is part detective, part planner, and part communicator. You'll spend as much time talking with product owners and developers as you spend running tests.



© 2026 by Innovation In Software Corporation

QA SKILLS AND MINDSET

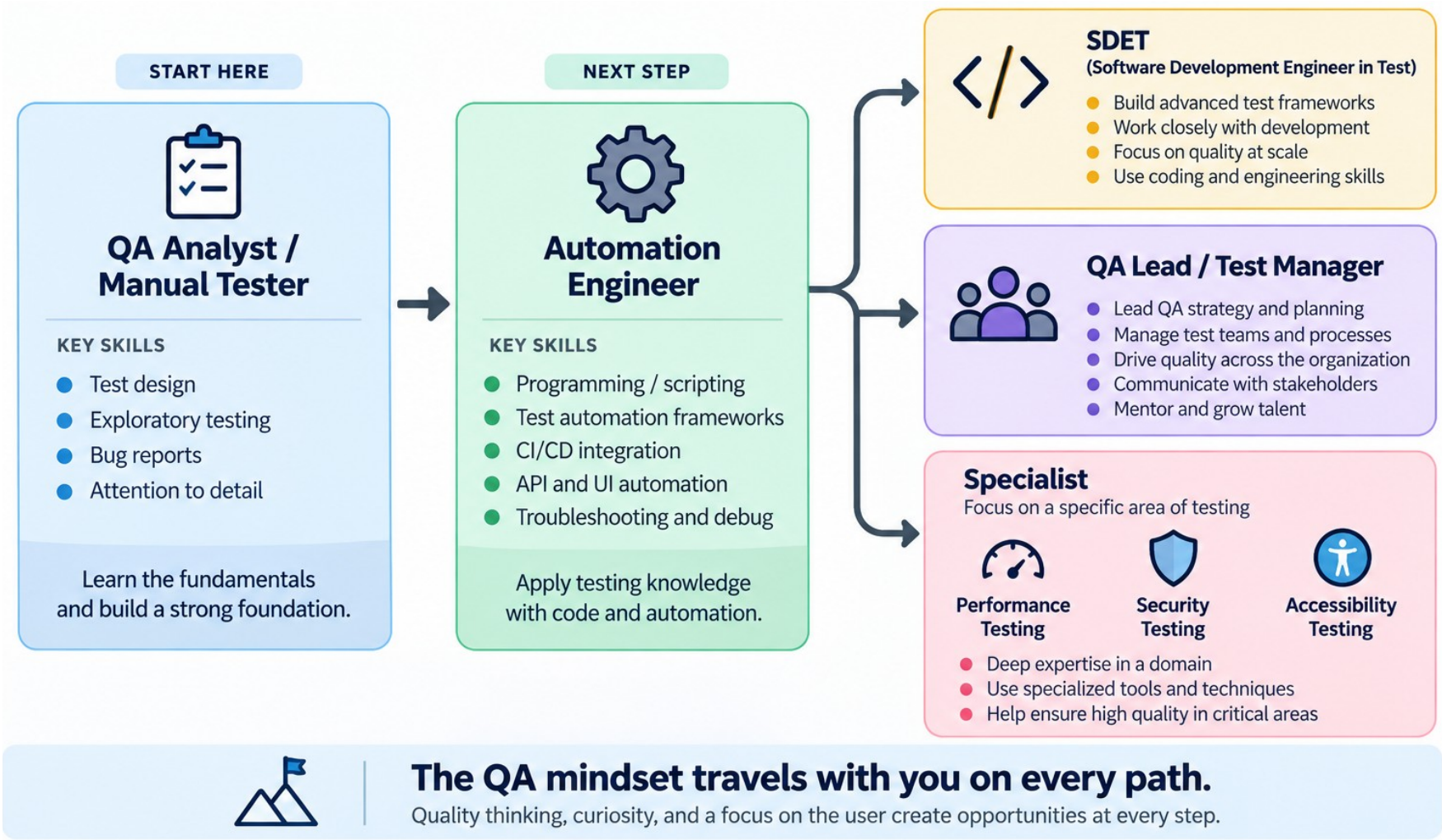
Tools change every few years. The QA mindset doesn't.

- **Curiosity:** "What happens if...?" is your favorite question
- **Skepticism:** "It works on my machine" is a claim, not evidence
- **Risk sense:** knowing where bugs are likely and where they would hurt
- **Communication:** reporting problems clearly, without blame
- **Attention to detail:** the exact message, the exact amount, the exact step



© 2026 by Innovation In Software Corporation

THE QA CAREER PATH



© 2026 by Innovation In Software Corporation

WATERFALL: TESTING AS A PHASE

In a Waterfall project, work flows through fixed phases in order: requirements, design, build, test, release. Testing is a phase near the end.

- Requirements are fixed up front, so test plans can be written early
- Testers often see working software only after development is "done"
- Defects found late are expensive, and schedule pressure squeezes the test phase



© 2026 by Innovation In Software Corporation

AGILE: TESTING EVERY SPRINT

Agile teams deliver in short iterations called sprints, usually one to four weeks. Each sprint delivers a small, working slice of the product.

- QA tests each user story inside the sprint it's built in
- Requirements evolve, so test plans are living documents
- Testers work side by side with developers and the product owner
- "Done" means tested, not just coded



© 2026 by Innovation In Software Corporation

THE TESTER ON AN AGILE TEAM

On an Agile team, QA isn't a separate department at the end of the line. The tester is part of the team from the first conversation about a story.

- **Sprint planning:** ask how each story will be tested and size the testing work
- **Daily stand-up:** raise blockers and bugs early
- **During the sprint:** test each story as it's built, not all at the end
- **Review and retrospective:** show what was tested and improve how the team works



© 2026 by Innovation In Software Corporation

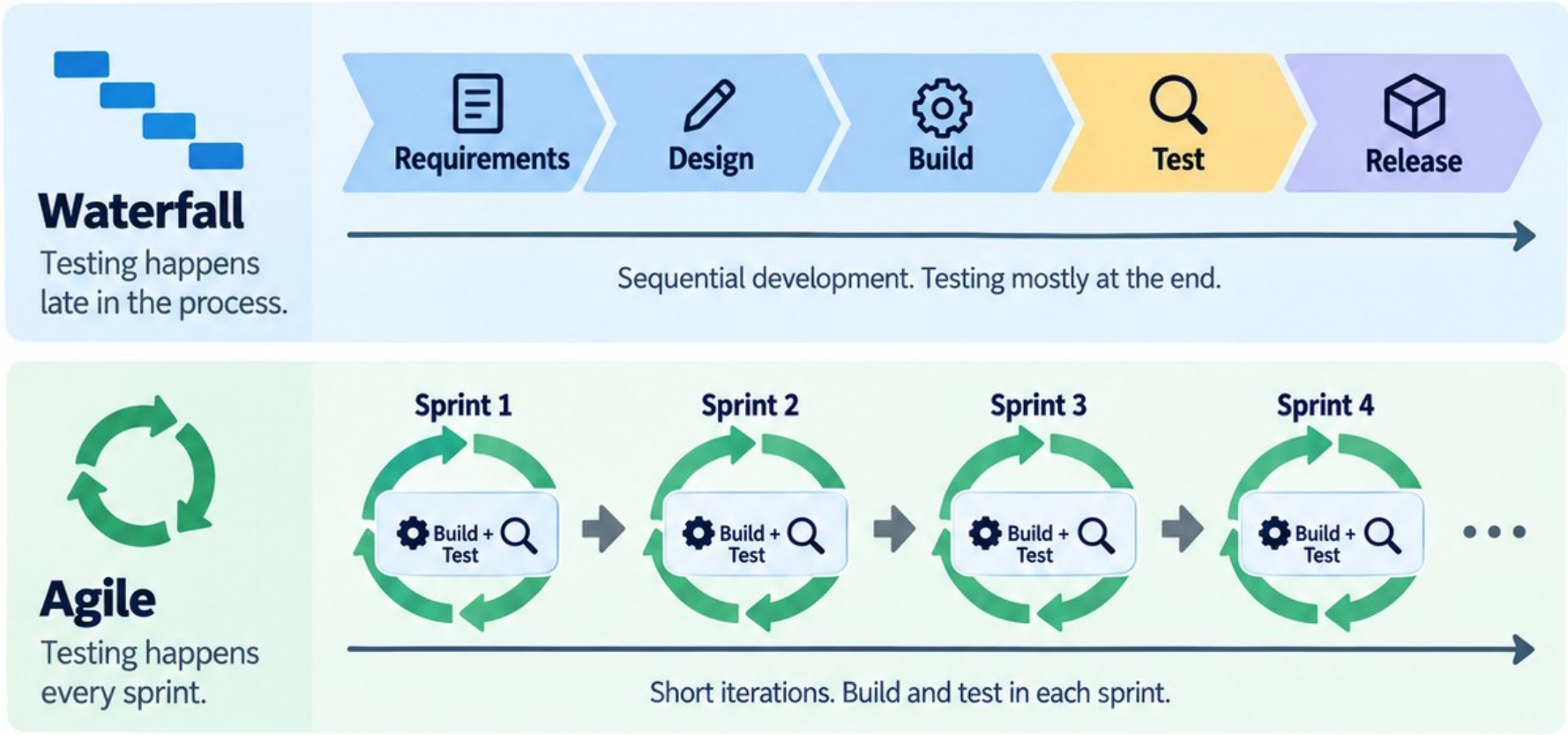
DEVOPS: TESTING ALL THE TIME

DevOps joins development and operations so teams can release small changes often and safely. Testing becomes continuous.

- Automated tests run on every change, in a pipeline
- QA designs the checks that decide whether a change can ship
- Fast feedback replaces long test phases
- Monitoring production is part of quality, too

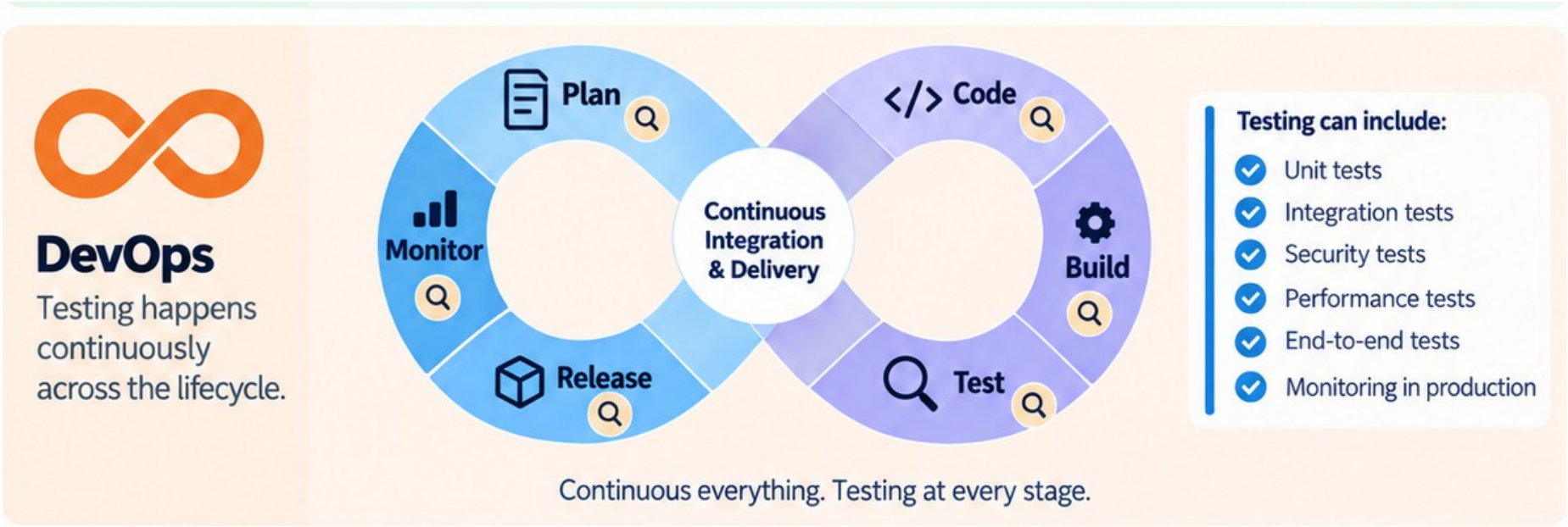
© 2026 by Innovation In Software Corporation

WHEN DOES TESTING HAPPEN? WATERFALL AND AGILE



© 2026 by Innovation In Software Corporation

WHEN DOES TESTING HAPPEN? DEVOPS



The methodology changes when testing happens,
not whether quality matters.

© 2026 by Innovation In Software Corporation

COMPARING THE THREE APPROACHES

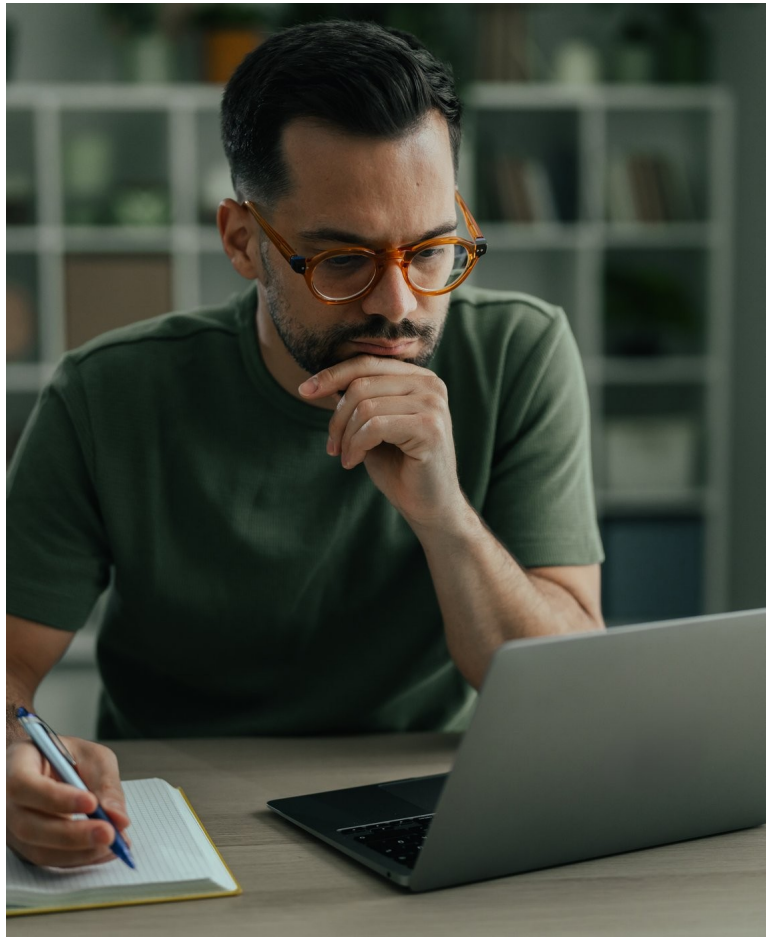
	Waterfall	Agile	DevOps
When testing happens	A phase near the end	Every sprint, story by story	Continuously, on every change
Requirements	Fixed up front	Evolve each sprint	Evolve continuously
QA's main role	Execute the test phase	Test inside the team	Design the automated checks and feedback
Feedback speed	Weeks or months	Days	Minutes
Biggest testing risk	Bugs found too late	Not enough time per sprint	Trusting automation that doesn't check enough

© 2026 by Innovation In Software Corporation

WHY PLAN YOUR TESTING?

You can never test everything. A login form alone has endless combinations of emails, passwords, and timing. A test plan decides what you will test, how deeply, and what you will leave out on purpose.

Without a plan, testing follows whatever is easiest or most recent. Important features get missed, effort is spread evenly across things that don't matter equally, and nobody can say when testing is done.



© 2026 by Innovation In Software Corporation

TEST STRATEGY VS. TEST PLAN

The two terms are related but answer different questions.

	Test strategy	Test plan
Answers	How does this organization test?	How will we test this release?
Lifespan	Long-term, changes rarely	One project or release
Covers	Test levels, tools, standards, environments	Scope, priorities, schedule, entry and exit criteria
Example	"Every API change gets automated tests in CI"	"Test checkout and discounts first for release 1.0"

© 2026 by Innovation In Software Corporation

WHAT GOES IN A TEST PLAN?

A good test plan is short. One page is often enough, and every section answers a question someone will ask.

- **Scope:** what's in, and what's explicitly out
- **Approach:** which kinds of testing apply to which features
- **Entry criteria:** what must be true before testing starts
- **Exit criteria:** what must be true before you recommend release
- **Environments and data:** where you test and with what
- **Risks and assumptions:** what could derail the testing itself

© 2026 by Innovation In Software Corporation

ENTRY AND EXIT CRITERIA

Entry and exit criteria turn "Are we ready?" into a checklist instead of a feeling.

- **Entry example:** the build is deployed to the test environment and test accounts exist
- **Exit example:** no open critical or high defects, and all top-priority tests pass
- Agree on them **before** testing starts, when nobody is under deadline pressure
- They protect the team: release decisions are made against criteria, not opinions



© 2026 by Innovation In Software Corporation

TESTABLE REQUIREMENTS

Good QA starts before any code exists. A requirement you can't test is a defect in the requirements.

Ask one question of every requirement: "How would I know this passed?" Watch for three warning signs.

- **Vague:** no measurable pass or fail, like "search should be fast"
- **Incomplete:** a missing value or rule, like a lockout with no duration
- **Conflicting or unclear:** rules that could interact in more than one way

Every problem you find becomes a question for the product owner, and every answer prevents a bug.

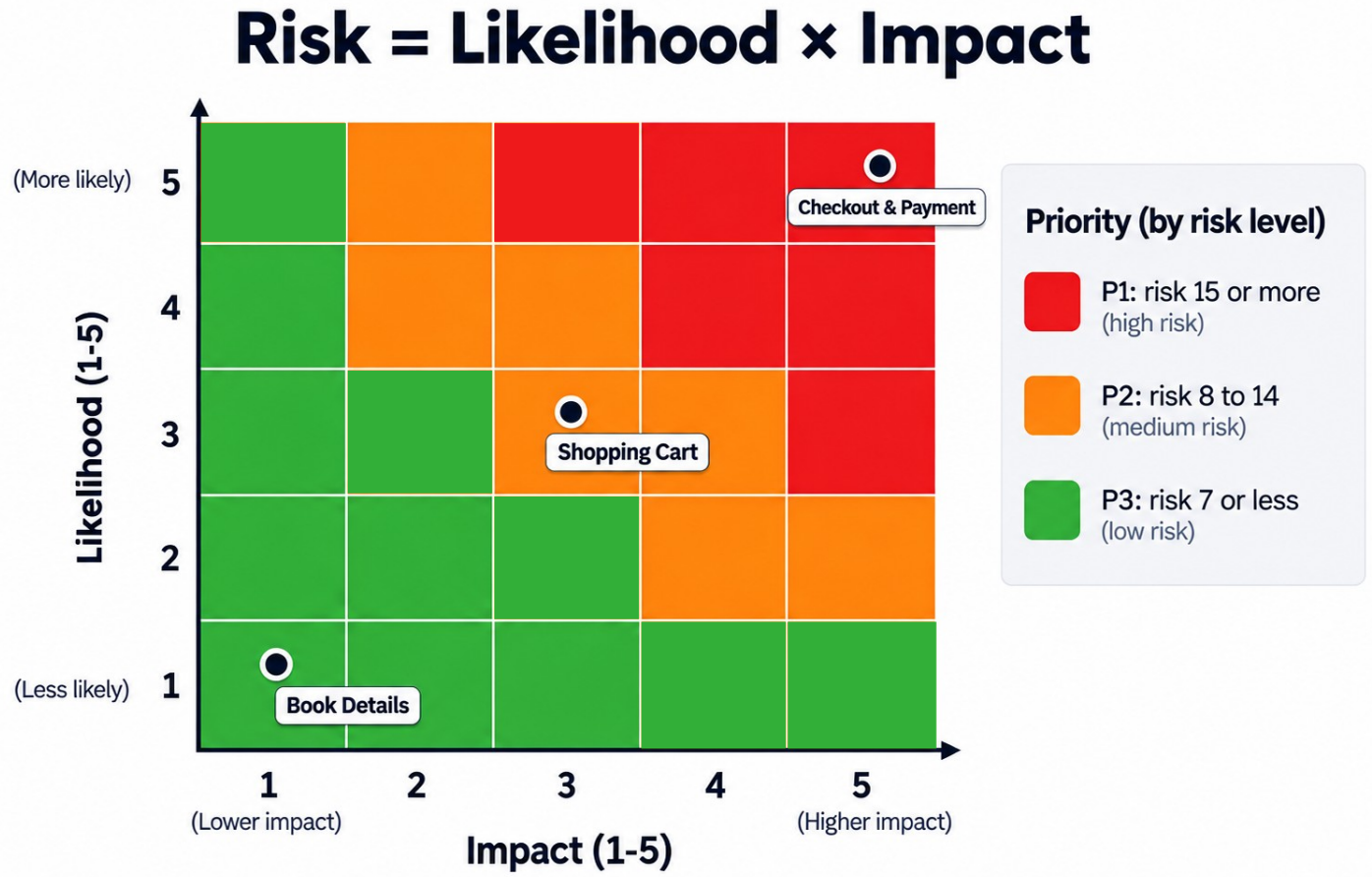
© 2026 by Innovation In Software Corporation

RISK-BASED TESTING

When time is limited, risk tells you where to look first. Risk combines two questions: how likely is this feature to break, and how bad would it be if it did?

Risk = Likelihood × Impact

- **Likelihood rises with:** complex rules, integrations, new or rushed code, vague requirements
- **Impact rises with:** money, personal data, security, number of users affected, reputation
- Score each from 1 to 5, multiply, and sort from highest to lowest



© 2026 by Innovation In Software Corporation

FROM RISK SCORE TO PRIORITY

A risk score is only useful if it changes what you do. Map score ranges to priorities, and priorities to how deeply you test.

Risk score	Priority	Test depth
15 to 25	**P1**	Test first: positive, negative, and edge cases
8 to 14	**P2**	Core flows plus key negative cases
1 to 7	**P3**	A quick smoke check, if time allows

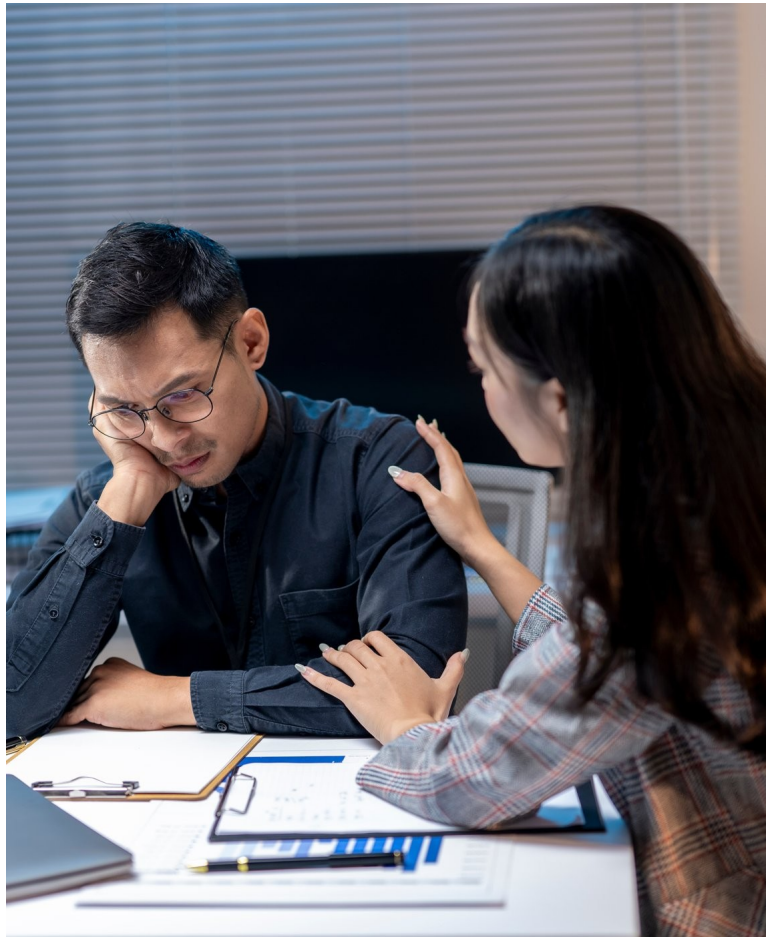
The reason behind a score matters more than the number. "Money changes hands and a third party is involved" is a reason a stakeholder can agree or argue with.

© 2026 by Innovation In Software Corporation

COMMON PLANNING PITFALLS

Most test plans fail in predictable ways. Knowing them is the first step to avoiding them.

- Spreading effort evenly instead of by risk
- Ignoring non-functional requirements like performance, security, and accessibility
- Never saying what's out of scope, so everything is implicitly promised
- Starting to test before the requirement questions are answered
- Writing a long plan nobody reads



© 2026 by Innovation In Software Corporation

LAB 1.1: TEST PLANNING

In this lab you join BookNook as its only QA engineer, four weeks before launch, and write the test plan that answers one question: "What will you test first, and why?"

- Review the BookNook requirements and record the vague, incomplete, and unclear ones
- Score every feature by likelihood and impact, and assign priorities
- Draft a one-page test plan with scope, approach, and entry and exit criteria
- Describe how your plan would change under Waterfall, Agile, and DevOps

© 2026 by Innovation In Software Corporation

POP QUIZ:

After several vague requirements caused bugs, a QA engineer adds an ‘Is this testable?’ check to the team’s requirements-review process. Which activity is this?

A. Quality assurance

B. Testing

C. Quality control

D. Regression testing



POP QUIZ:

After several vague requirements caused bugs, a QA engineer adds an ‘Is this testable?’ check to the team’s requirements-review process. Which activity is this?

A. Quality assurance

B. Testing

C. Quality control

D. Regression testing



POP QUIZ:

A feature has a likelihood score of 4 and an impact score of 5. What is its risk score, and what priority does it get?

A. 9, P2

B. 20, P1

C. 20, P2

D. 9, P3



POP QUIZ:

A feature has a likelihood score of 4 and an impact score of 5. What is its risk score, and what priority does it get?

A. 9, P2

B. 20, P1

C. 20, P2

D. 9, P3



POP QUIZ:

In which methodology do automated tests run on every change as part of a pipeline?

A. Waterfall

B. Agile, but only at the end of each sprint

C. DevOps

D. None; tests always run in a separate phase





POP QUIZ:

In which methodology do automated tests run on every change as part of a pipeline?

A. Waterfall

B. Agile, but only at the end of each sprint

C. DevOps

D. None; tests always run in a separate phase





TWO WAYS TO TEST

Once you know **what** to test, you still have to decide **how**. Testers use two complementary approaches.

Scripted testing follows test cases written in advance: exact steps and exact expected results. **Exploratory testing** designs the tests while running them, guided by what you learn as you go.

Neither is "better." They find different kinds of problems, and strong teams use both.

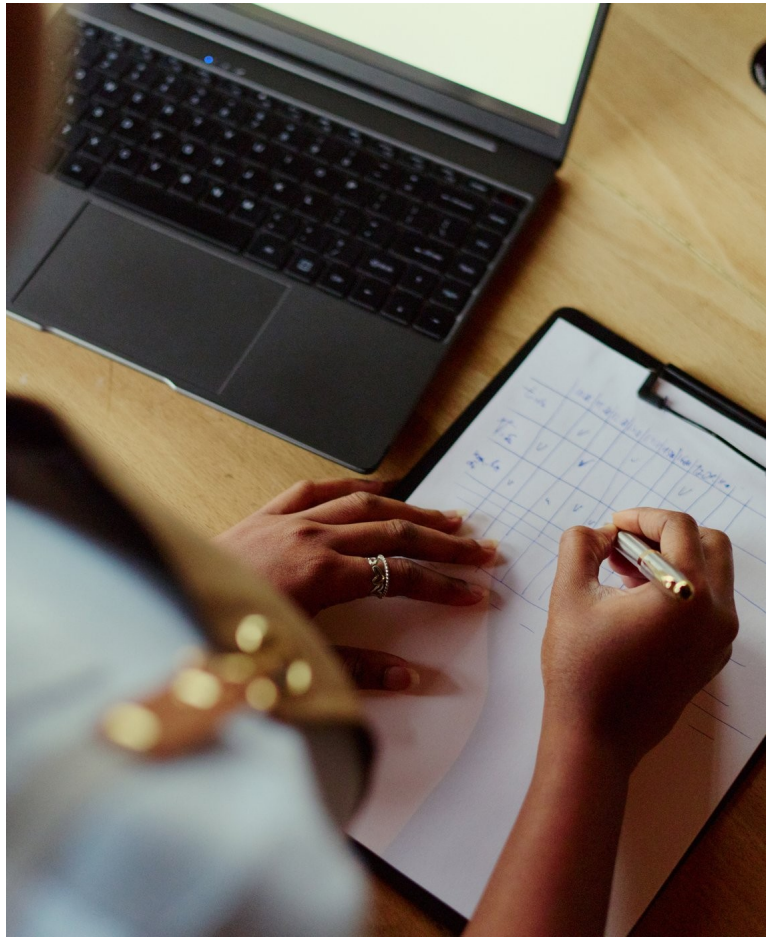


© 2026 by Innovation In Software Corporation

SCRIPTED TESTING

A scripted test case spells out every step and the exact result to expect. Anyone on the team can run it and get the same answer.

- **Repeatable:** the same steps every time, by anyone
- **Easy to delegate and automate:** the expected result is already written down
- **Great for regression:** proves that what worked yesterday still works today
- **Limited:** it only checks what someone thought of in advance



© 2026 by Innovation In Software Corporation

RUNNING A TEST: EXECUTION AND RESULTS

Executing a test means following the steps, comparing what happens with the expected result, and recording the outcome honestly.

Status	Means
Pass	The actual result matched the expected result exactly
Fail	It didn't; record what actually happened
Blocked	You couldn't run it (environment down, an earlier bug in the way)
Skipped	Deliberately not run this time, with a reason

When a bug is fixed, **retest** the failed case, then check the area around it.

© 2026 by Innovation In Software Corporation

EXPLORATORY TESTING

Exploratory testing is simultaneous learning, test design, and test execution. You try something, observe what happens, and let the result decide your next move.

It isn't random clicking. A skilled explorer works from a clear mission, uses proven techniques, and keeps notes. It's at its best on new features, high-risk areas, and requirements that are vague or incomplete.



© 2026 by Innovation In Software Corporation

SCRIPTED VS. EXPLORATORY

	Scripted	Exploratory
Tests designed	Before execution	During execution
Guided by	Step-by-step test cases	A **charter**: a mission and a timebox
Strengths	Repeatable, easy to delegate, good for regression	Finds the unexpected, adapts to what you learn
Weaknesses	Only checks what someone predicted	Harder to repeat; depends on tester skill
Typical use	Smoke and regression checks, compliance	New features, high-risk areas, vague requirements

© 2026 by Innovation In Software Corporation

THE EXPLORATORY SESSION

Exploratory testing works best in short, focused sessions. A **charter** gives the session a mission: *explore* a target, *with* certain techniques or resources, *to discover* a kind of problem.

- Set a **timebox**, often 20 to 90 minutes, and stop when it ends
- Stay on charter; jot down anything off-mission for later
- Log bugs with enough detail to reproduce them
- Log **questions** too: an unclear requirement is a finding

© 2026 by Innovation In Software Corporation

EXPLORATORY HEURISTICS

Heuristics are rules of thumb that suggest what to try next when you're not sure where to look.

Heuristic	Try this
Goldilocks	Too big, too small, just right
Zero, one, many	No items, one item, lots of items
CRUD	Create, read, update, and delete everything you can
Interrupt	Refresh, press Back, open a second tab, log out mid-task
Follow the data	Enter something on one page; check it everywhere it appears
Starve it	Leave fields blank; remove things it expects to exist

© 2026 by Innovation In Software Corporation

NOTES ARE EVIDENCE

Exploratory testing is hard to repeat unless you write things down. Short notes, taken as you go, turn a session into evidence someone else can follow.

- What you tried, in the order you tried it
- What you saw, especially anything surprising
- Where you stopped, so the next session can pick up



© 2026 by Innovation In Software Corporation

THE SESSION DEBRIEF

A short debrief turns one tester's session into team knowledge. Five minutes with a lead or a teammate is enough.

- What did you cover, and what didn't you get to?
- What bugs and questions did you find?
- What surprised you?
- Where should the next session go?

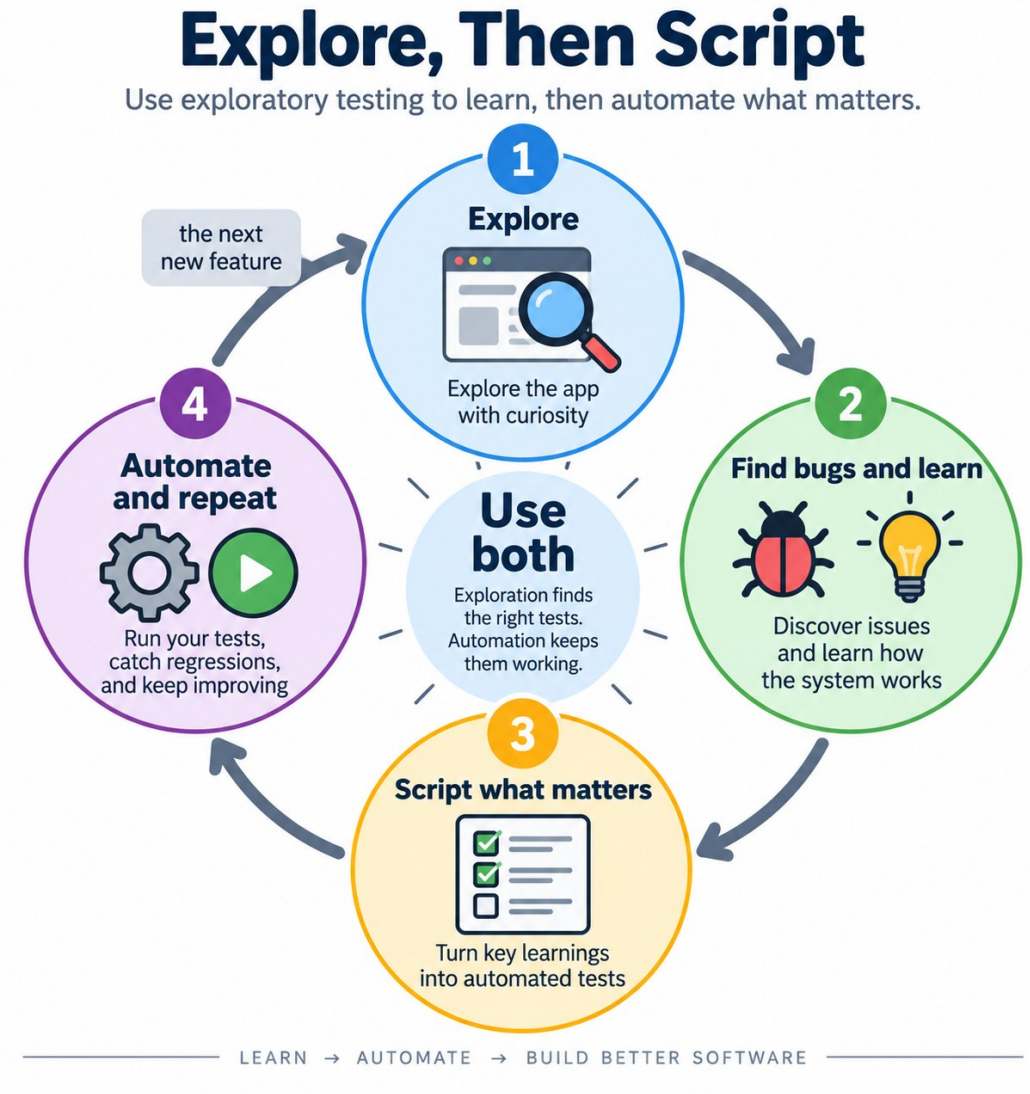


© 2026 by Innovation In Software Corporation

BETTER TOGETHER

Exploration and scripts feed each other. Explore to discover how the software really behaves, then turn the findings that matter into scripted tests so they stay fixed.

- Explore new and risky features first
- Script the important findings as regression tests
- Automate the scripts that run most often
- Free up people to explore the next new thing



© 2026 by Innovation In Software Corporation

WHERE BUGS HIDE

Most tests check the **happy path**: valid input, in the expected order. **Positive tests** like these prove a feature can work. **Negative tests** prove it fails safely when something goes wrong.

Bugs cluster at the edges: at limits, in bad input, and where features interact. Developers usually test the happy path. Much of QA's value is in testing everything else.



© 2026 by Innovation In Software Corporation

EQUIVALENCE PARTITIONING

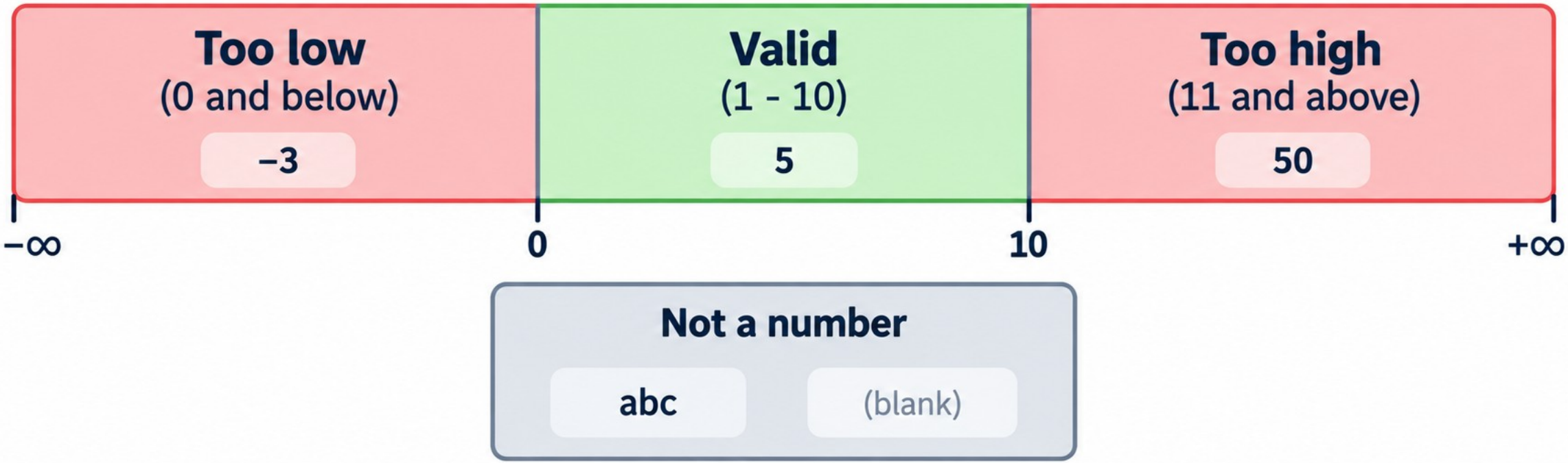
You can't test every value, but you don't need to. Inputs fall into **groups** the software should treat the same way. Test one value from each group.

For "quantity is 1 to 10," there are four groups: too low, valid, too high, and not a number. Four tests cover what thousands of values would.

© 2026 by Innovation In Software Corporation

EQUIVALENCE PARTITIONING

Divide the input range into partitions and test one value from each partition.



One test per partition, then add boundary values at the edges.

© 2026 by Innovation In Software Corporation

BOUNDARY VALUE ANALYSIS

When a rule has a limit, test **at** the limit and **just either side** of it. Off-by-one mistakes, like using > instead of >=, live exactly there.

For a rule that says "quantity is 1 to 10," the most valuable tests are 0, 1, 10, and 11, not 5.

© 2026 by Innovation In Software Corporation

MORE NEGATIVE AND EDGE-CASE TECHNIQUES

Technique	Ask yourself	Example
Invalid input	What if it's the wrong type, or empty?	Letters, blanks, negatives, decimals
Variations	Does it accept input that's "close enough"?	Upper/lower case, extra spaces
Combinations	What if two features interact?	Two discount codes; a code plus a quantity change
State and order	What if I do things in a different order?	Apply a code, *then* change the cart; refresh the page
Consistency	Does every page follow the same rule?	A rule enforced on one page but not another

© 2026 by Innovation In Software Corporation

THE COMBINATIONS PROBLEM

Features interact, and the number of combinations explodes fast. Three browsers, two login states, four discount codes, and five quantities already make 120 combinations, before anyone changes the order of steps.

- You can't test them all, so choose
- Focus on combinations where rules **interact**, like a discount plus a quantity change
- Use risk to decide which combinations matter most
- Pairwise testing covers every **pair** of values with far fewer tests

© 2026 by Innovation In Software Corporation

ERROR GUESSING

Experienced testers develop a nose for bugs. **Error guessing** means using that experience to target the mistakes developers commonly make.

- Off-by-one limits (> instead of >=)
- Text that isn't trimmed or compared case-insensitively
- Rules checked on one page but not another
- Money rounding, and totals that go negative
- Empty, missing, or duplicate data



© 2026 by Innovation In Software Corporation

FUNCTIONAL VS. NON-FUNCTIONAL TESTING

Functional testing checks **what** the software does: does it follow its rules? **Non-functional testing** checks **how well** it does it: how fast, how secure, how easy to use, how dependable.

A feature can pass every functional test and still fail its users. A correct search that takes 20 seconds, or a checkout a keyboard user can't reach, is still broken.

© 2026 by Innovation In Software Corporation

FUNCTIONAL VS. NON-FUNCTIONAL TESTING

Functional

The things the system is supposed to do.



- Adds items
- Applies discount
- Calculates total

Non-Functional

How well the system does it.



Performance
Fast and responsive.



Security
Keeps data safe.



Usability
Easy and pleasant to use.



Reliability
Available when needed.

© 2026 by Innovation In Software Corporation

TYPES OF NON-FUNCTIONAL TESTING

Type	Question it answers	BookNook requirement
Performance	Is it fast enough, under real load?	NF1: pages load in under 3 s for 95% of requests with 200 users
Security	Can it be misused or broken into?	NF2: passwords hashed, HTTPS everywhere
Usability	Can real people use it easily?	NF5: "The site should be user-friendly."
Reliability	Does it keep working over time?	NF6: 99.5% uptime during business hours

Look closely at NF5. How would you know it passed?

© 2026 by Innovation In Software Corporation

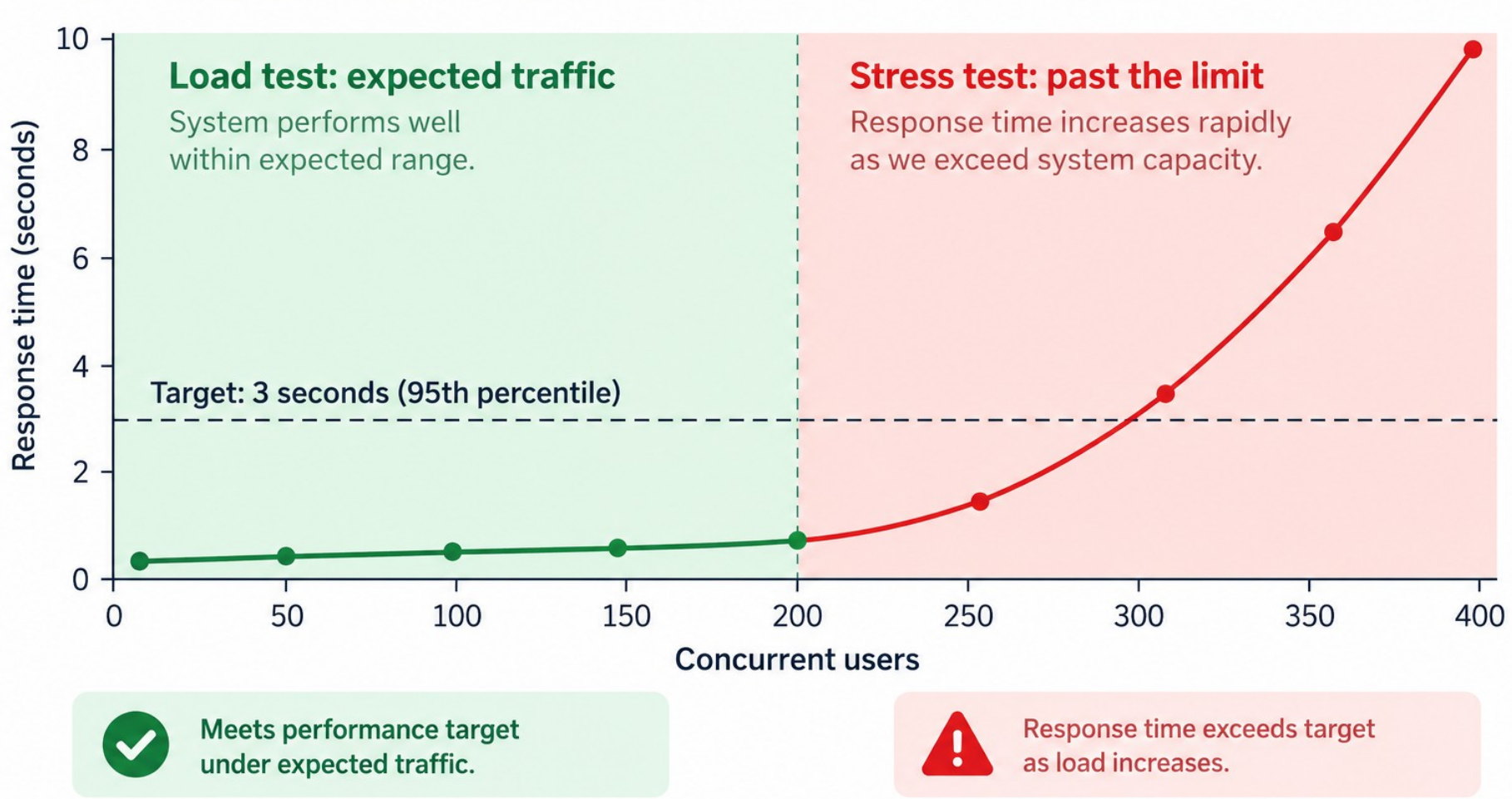
PERFORMANCE TESTING

Performance testing checks whether the system is fast enough under realistic and heavy use, not just with one tester clicking around.

- **Load testing:** expected traffic, such as 200 users at once
- **Stress testing:** beyond the limit, to see how it fails
- **Measure percentiles:** "95% of pages under 3 seconds" says more than an average
- Use dedicated tools and a production-like environment

© 2026 by Innovation In Software Corporation

PERFORMANCE TESTING



© 2026 by Innovation In Software Corporation

SECURITY TESTING

Security testing looks for ways the system can be misused, broken into, or made to leak data. Much of it is specialist work, but every tester can check the basics.

- Error messages that reveal too much (for example, "no account with that email")
- Limits on repeated attempts, like account lockout
- Data that should be private, such as other users' carts
- Sessions that end when they should

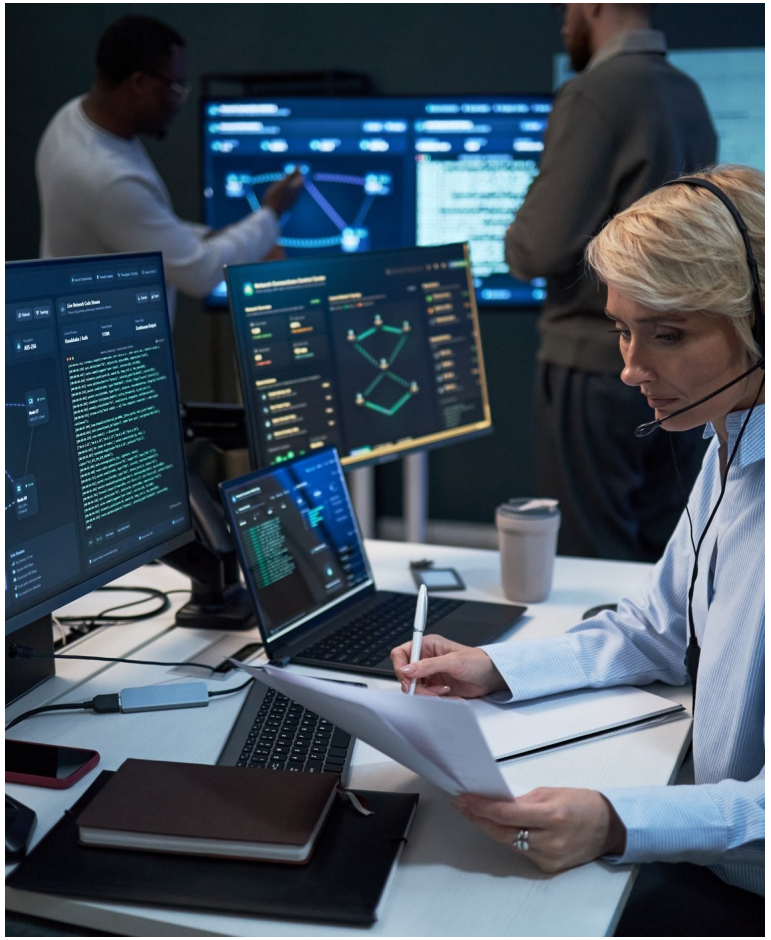


© 2026 by Innovation In Software Corporation

RELIABILITY TESTING

Reliability testing checks whether the system keeps working over time and recovers when something goes wrong.

- Does it stay up for hours or days under steady use?
- When one request fails, do others keep working?
- After a crash or restart, does it come back cleanly, with data intact?
- Requirements state reliability as targets, like "99.5% uptime"



© 2026 by Innovation In Software Corporation

USABILITY TESTING

Usability testing watches **real people** try to do real tasks. It finds the problems people who built the product are too close to see.

- Give a user a task, like "buy a book with a discount code"
- Watch where they hesitate, misclick, or give up
- Measure: did they finish, how long it took, how many errors they made
- Turn vague goals like "user-friendly" into measurable ones



© 2026 by Innovation In Software Corporation

USABILITY BUGS ARE REAL BUGS

Some bugs break no written rule. The feature works, but a real customer would be confused, frustrated, or stuck.

- A technical error message shown to a shopper
- A rule that rejects input a person would reasonably type
- A button that does something different from what its label says

Log these as **usability** defects. They cost sales just like functional ones.



© 2026 by Innovation In Software Corporation

CLASSIFYING WHAT YOU FIND

Classifying each defect helps the team see patterns and decide who should act.

Type	Means	Example
Functional	Wrong behavior according to a requirement	A rule from the spec isn't enforced
Usability	Works, but confusing or unfriendly	A raw technical message instead of a helpful one
Performance	Correct, but too slow under load	Search takes 20 seconds with 200 users
Security	Can be misused or leaks data	Another user's cart is visible
Reliability	Fails or loses data over time or after errors	The cart empties after a server restart

© 2026 by Innovation In Software Corporation

A PASS CAN STILL MISS THE BUG

A scripted test is only as good as its expected result. "An error message is displayed" passes for *any* message, even `Error: null`.

- Write expected results that are specific: the exact text, the exact amount
- A loose expected result turns a real bug into a green checkmark
- When a passing result looks wrong to you, say so



© 2026 by Innovation In Software Corporation

LAB 1.2: SCRIPTED VS. EXPLORATORY TESTING

In this lab you test BookNook's cart and discount codes two ways, then compare what each approach found.

- Run a developer's nine-step scripted checklist and record pass or fail
- Run a 20-minute exploratory session with a charter, using boundary values, invalid input, and combinations
- Log every bug and question with steps to reproduce
- Classify each bug and decide which approach found it

© 2026 by Innovation In Software Corporation

POP QUIZ:

A requirement says quantity must be between 1 and 10. Which set of values gives the best boundary value test?

A. 2, 5, 8

B. 0, 1, 10, 11

C. 1, 5, 10

D. 5, 50, 500



POP QUIZ:

A requirement says quantity must be between 1 and 10. Which set of values gives the best boundary value test?

A. 2, 5, 8

B. 0, 1, 10, 11

C. 1, 5, 10

D. 5, 50, 500



POP QUIZ: What guides an exploratory testing session?

A. A step-by-step script with expected results

B. Random clicking until something breaks

C. The developer's unit tests

D. A charter with a mission and a timebox





POP QUIZ: What guides an exploratory testing session?

A. A step-by-step script with expected results

B. Random clicking until something breaks

C. The developer's unit tests

D. A charter with a mission and a timebox





POP QUIZ:

Search returns the correct books, but takes 20 seconds when 200 customers search at once. What kind of defect is this?

A. Functional

B. Non-functional: performance

C. Non-functional: usability

D. Not a defect, because the results are correct





POP QUIZ:

Search returns the correct books, but takes 20 seconds when 200 customers search at once. What kind of defect is this?

A. Functional

B. Non-functional: performance

C. Non-functional: usability

D. Not a defect, because the results are correct



FROM REQUIREMENTS TO USER STORIES

In Lab 1.1 you tested a requirements document. Agile teams usually describe work as **user stories** instead: short descriptions of something a user needs, written from that user's point of view.

A story is small enough to build and test in one sprint. It's also a promise of a conversation. The details are worked out together by the product owner, developers, and QA before the work starts.



© 2026 by Innovation In Software Corporation

ANATOMY OF A USER STORY

Most stories follow one simple pattern:

As a *type of user*, **I want** *some goal* **so that** *some benefit*.

- **Who:** the user, which tells you whose experience to test
- **What:** the goal, which is the behavior under test
- **Why:** the benefit, which helps you judge what "good enough" means

As a shopper, I want to apply a discount code in my cart so that I pay less.

© 2026 by Innovation In Software Corporation

INVEST: A TESTER'S CHECKLIST FOR STORIES

INVEST is a checklist for well-formed user stories. Testers care most about the last letter, but every letter affects testing.

Letter	Means	Why QA cares
Independent	Can be built and tested alone	No waiting on another story to test this one
Negotiable	Details worked out together	Your questions shape the criteria
Valuable	Delivers value to a user	Tells you what "good enough" means
Estimable	The team can size it	Testing effort is part of the estimate
Small	Fits in one sprint	Small stories mean focused tests
Testable	Has clear pass/fail criteria	No criteria, no test

© 2026 by Innovation In Software Corporation

THE THREE AMIGOS

Before a story is built, three perspectives review it together: the **product owner** (what's needed), a **developer** (how to build it), and **QA** (what could go wrong).

A 15-minute conversation surfaces missing rules, edge cases, and misunderstandings while they're still cheap to fix. The acceptance criteria often come straight out of it.

© 2026 by Innovation In Software Corporation

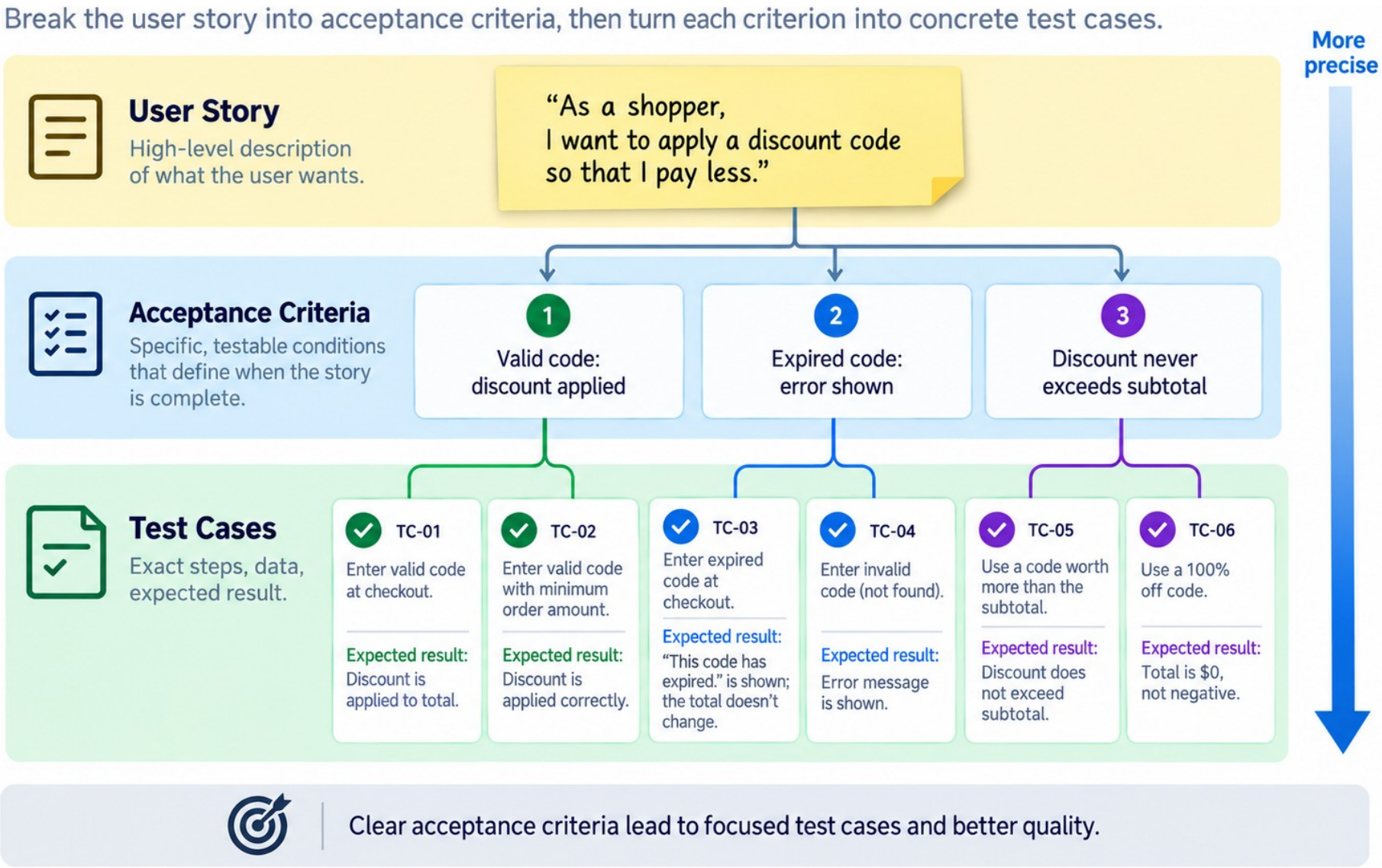
FROM STORY TO TEST CASE

Each layer adds precision. The story says what the user wants. Acceptance criteria say when the story is done. Test cases say exactly how to check each criterion.



© 2026 by Innovation In Software Corporation

FROM STORY TO TEST CASE



© 2026 by Innovation In Software Corporation

STORIES, CRITERIA, TEST CASES, AND DONE

Artifact	Answers	Example
User story	Who wants what, and why?	*As a shopper, I want to apply a discount code so that I pay less.*
Acceptance criteria (AC)	When is **this story** done?	*Given an expired code, when I apply it, then I see "This code has expired."*
Test case	Exactly how do I check one AC?	Preconditions, data, steps, expected result
Definition of Done (DoD)	When is **any** story done?	Code reviewed, all AC tests pass, no open critical bugs

© 2026 by Innovation In Software Corporation

ACCEPTANCE CRITERIA: GIVEN, WHEN, THEN

Given / When / Then gives every criterion the same clear shape: a starting state, an action, and a result you can check.

- **Given** the context: *a cart with a $40.00 subtotal*
- **When** the action: *the shopper applies an expired code*
- **Then** the observable result: *they see "This code has expired."*

If you can't say what "then" looks like, the criterion isn't testable yet. Take it back to the product owner.

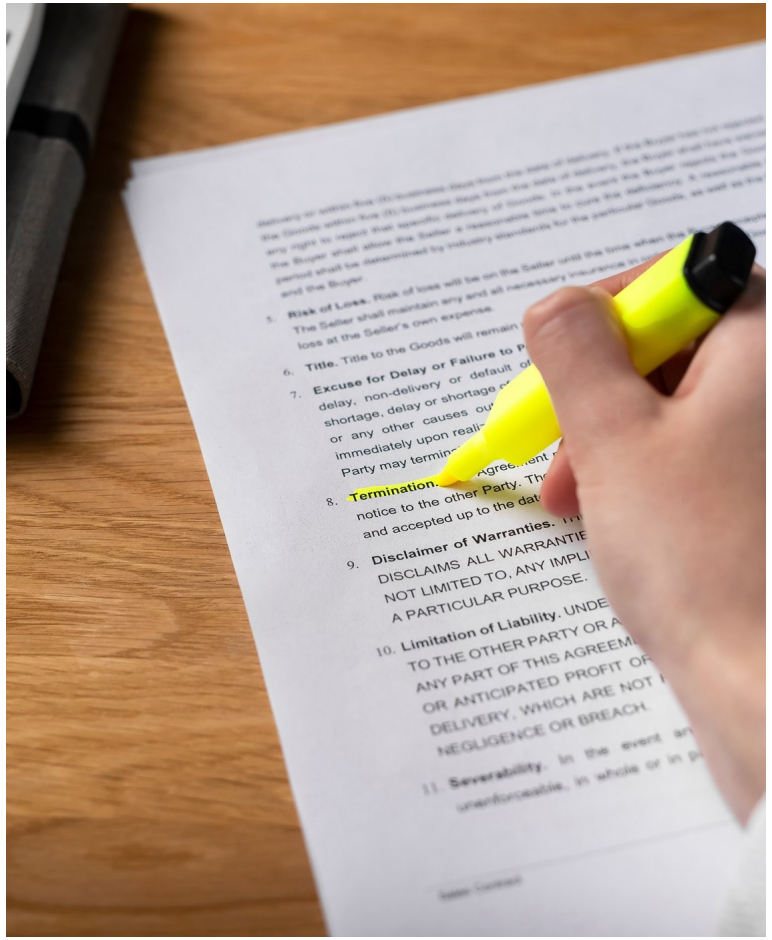


© 2026 by Innovation In Software Corporation

WRITING STRONG ACCEPTANCE CRITERIA

Good acceptance criteria are short, specific, and checkable as pass or fail.

- One behavior per criterion
- Exact messages, amounts, and limits instead of "an error" or "fast"
- Cover what should **not** happen too: invalid input, wrong state, limits
- Use the product owner's answers to your questions, not your own guesses



© 2026 by Innovation In Software Corporation

THE DEFINITION OF DONE

Acceptance criteria are different for every story. The **Definition of Done** is one shared checklist the team applies to **every** story before calling it done.

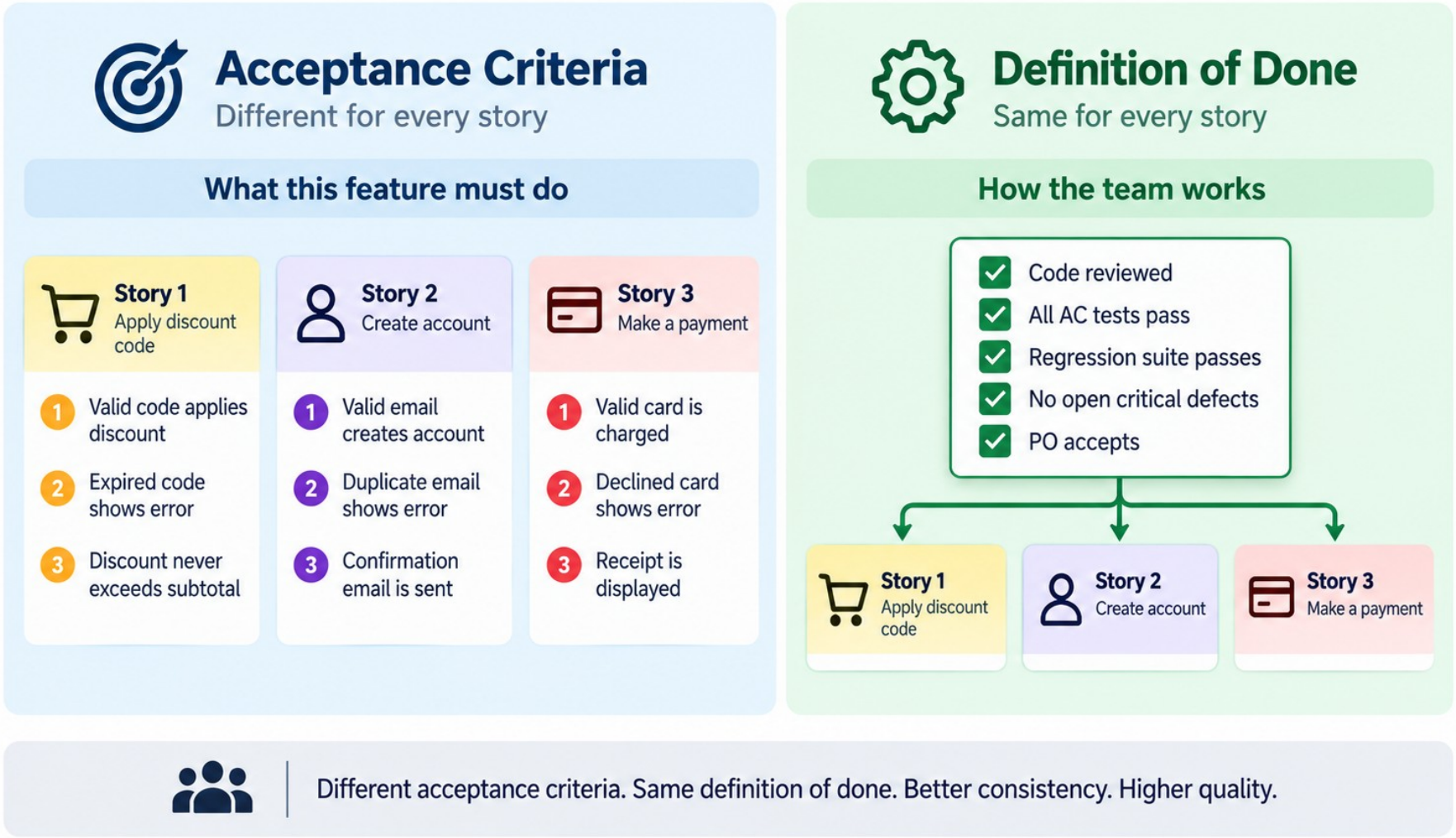
A typical DoD mixes testing with other quality steps: code reviewed and merged, every acceptance criterion has a passing test, the regression suite passes, no open critical defects, and the product owner accepts the story.

Some of these items are proven by test cases. Others are proven by reviews, tools, or people.



© 2026 by Innovation In Software Corporation

ACCEPTANCE CRITERIA VS. DEFINITION OF DONE



© 2026 by Innovation In Software Corporation

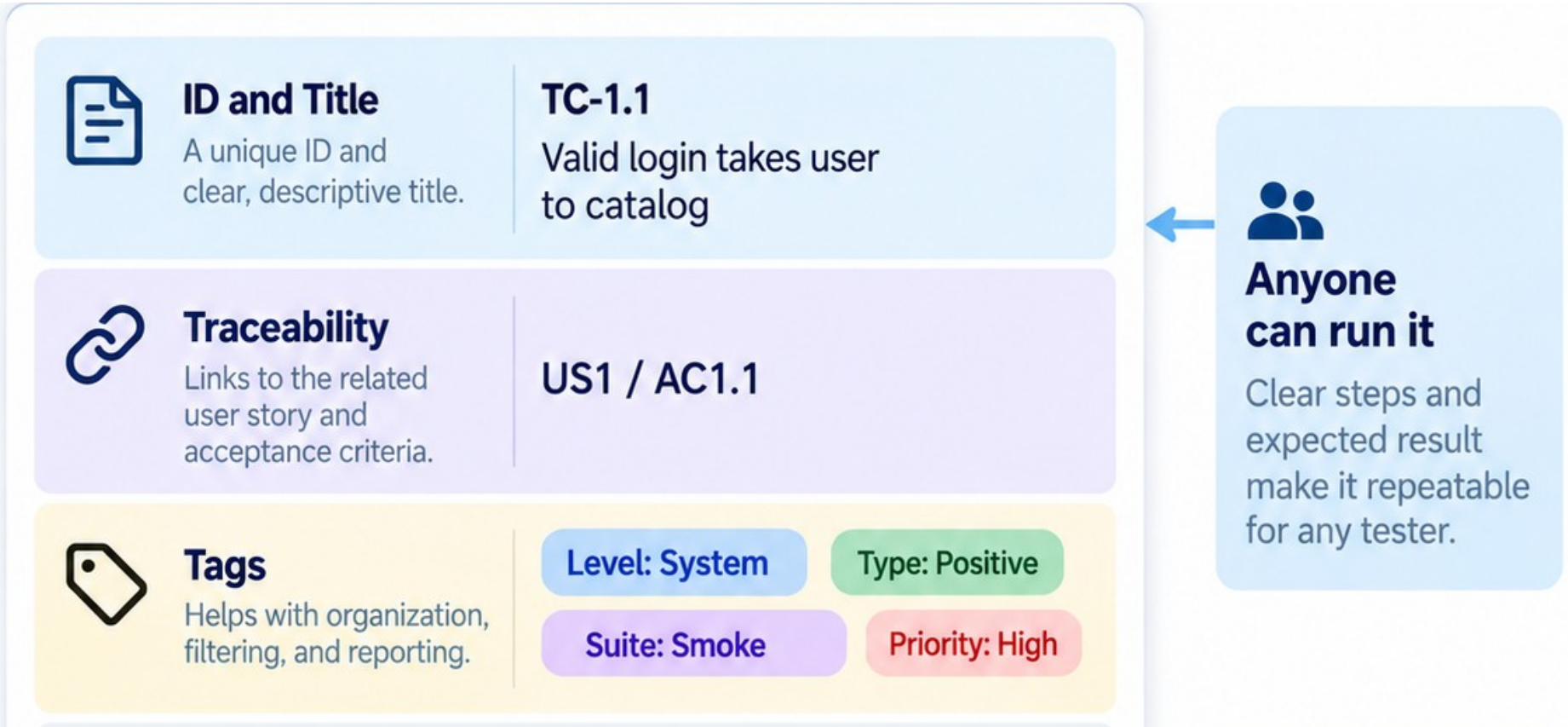
WHAT MAKES A GOOD TEST CASE

A test case is a recipe someone else can follow without asking you a single question.

Quality	Means	Avoid
Clear	Anyone on the team can run it	"Check the discount works"
Concise	One purpose per test case	Ten checks in one test
Specific	Exact data and exact expected results	"An error is shown"
Reusable	Shared preconditions; data in a table	Copy-pasting the same steps for every value
Traceable	Points to the acceptance criterion it verifies	Orphan tests nobody can explain

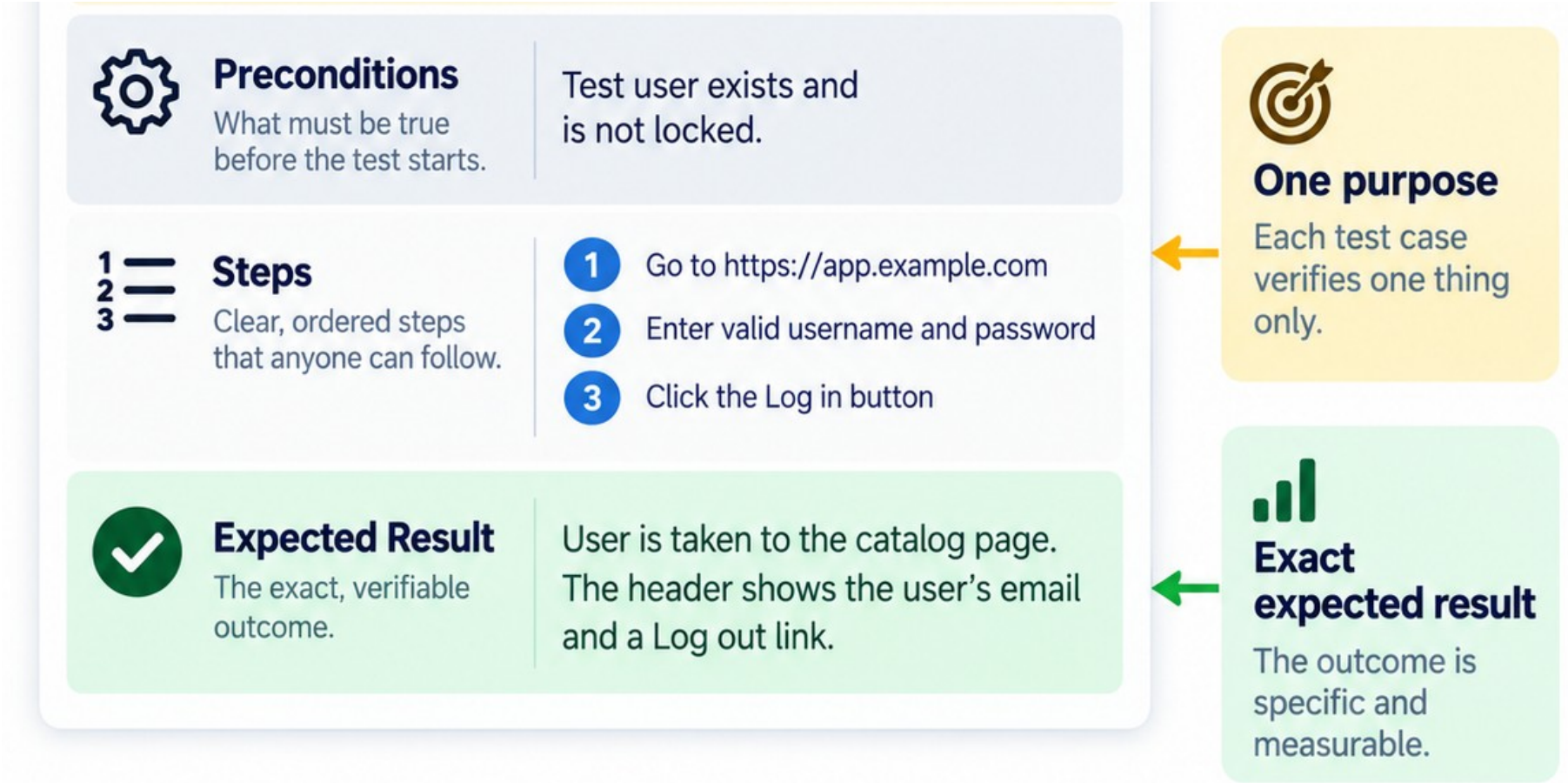
© 2026 by Innovation In Software Corporation

ANATOMY OF A TEST CASE



© 2026 by Innovation In Software Corporation

ANATOMY OF A TEST CASE



© 2026 by Innovation In Software Corporation

REUSABLE TEST CASES WITH DATA TABLES

When several values share the same steps, write the steps once and put the values in a table. One test case then covers a whole boundary.

Failed attempts	Then log in correctly	Expected
4	Yes	Login succeeds; the count resets
5	Yes	"Account locked. Try again in 15 minutes."

Testing both sides of the limit, 4 and 5, takes one test case, not two. Data tables also make test cases easy to automate later.



© 2026 by Innovation In Software Corporation

POSITIVE, NEGATIVE, AND EDGE CASES

A strong set of test cases for one story covers more than the happy path.

- **Positive:** valid input, expected flow (a valid code applies the discount)
- **Negative:** invalid input or the wrong state (an unknown code, an expired code)
- **Edge:** values at or near a limit (a discount larger than the cart subtotal)

Use the techniques from Lab 1.2: boundaries, invalid input, variations, and combinations.



© 2026 by Innovation In Software Corporation

TEST LEVELS

Tests run at different levels, from one small piece of code up to the whole business need.

Level	Checks	BookNook example
Unit	One function in isolation	`calculateDiscount(7998, "SAVE15")` returns 1200
Integration	Components working together	The cart API reads book prices from the catalog correctly
System	The whole app, end to end	Add to cart, apply a code, see the right total in the browser
Acceptance	The story meets the business need	The product owner confirms the discount rules match the story

Today's test cases are mostly **system** and **acceptance** level. Tagging the level now helps decide what to automate tomorrow.

© 2026 by Innovation In Software Corporation

UNIT TESTING

Unit tests check the smallest pieces of code, like a single function, in isolation. Developers usually write them, and they run in milliseconds.

- Fast enough to run on every change
- Pinpoint exactly which function broke
- Can't see problems between components or in the user interface

Tomorrow you'll write unit tests yourself, before writing the code they test.



© 2026 by Innovation In Software Corporation

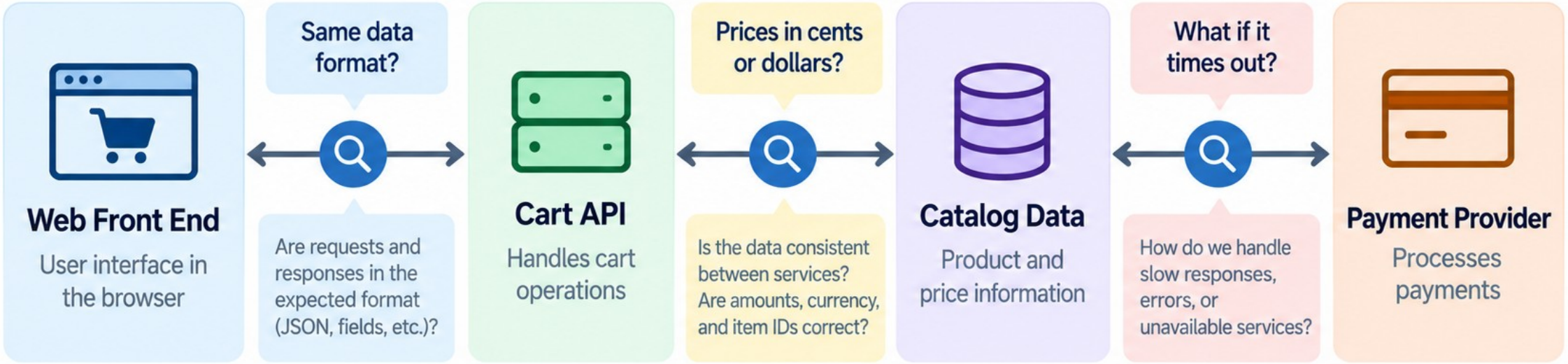
INTEGRATION TESTING

Integration tests check that components work **together**: the front end and the API, the API and the database, your system and a payment provider.

Bugs here hide in the gaps: mismatched data formats, wrong units (cents vs. dollars), or a field one side sends and the other ignores.

© 2026 by Innovation In Software Corporation

INTEGRATION TESTING



Each part can work alone and still fail together.
That's why we test the connections.

© 2026 by Innovation In Software Corporation

SYSTEM TESTING

System testing checks the whole application end to end, the way a user would use it, against the requirements.

- Runs on a complete, deployed build in a test environment
- Covers functional and non-functional requirements
- Most of today's test cases are system tests: real pages, real flows, real data

© 2026 by Innovation In Software Corporation

ACCEPTANCE TESTING AND UAT

Acceptance testing answers the final question: does this meet the business need? **User acceptance testing (UAT)** puts the software in front of real users or the product owner before release.

- Based on acceptance criteria and real business scenarios
- Run by people who know the business, not only by QA
- Passing UAT is often a release gate



© 2026 by Innovation In Software Corporation

TEST SUITES: SMOKE, SANITY, REGRESSION

A suite is a group of test cases run together for a purpose. One test case can belong to several suites.

Suite	When it runs	Scope
Smoke	Every new build or deploy	Broad and shallow: a few critical happy paths
Sanity	After a small fix or change	Narrow and focused on that change and what's right next to it
Regression	Before every release, or on every change	Everything that used to work still works

© 2026 by Innovation In Software Corporation

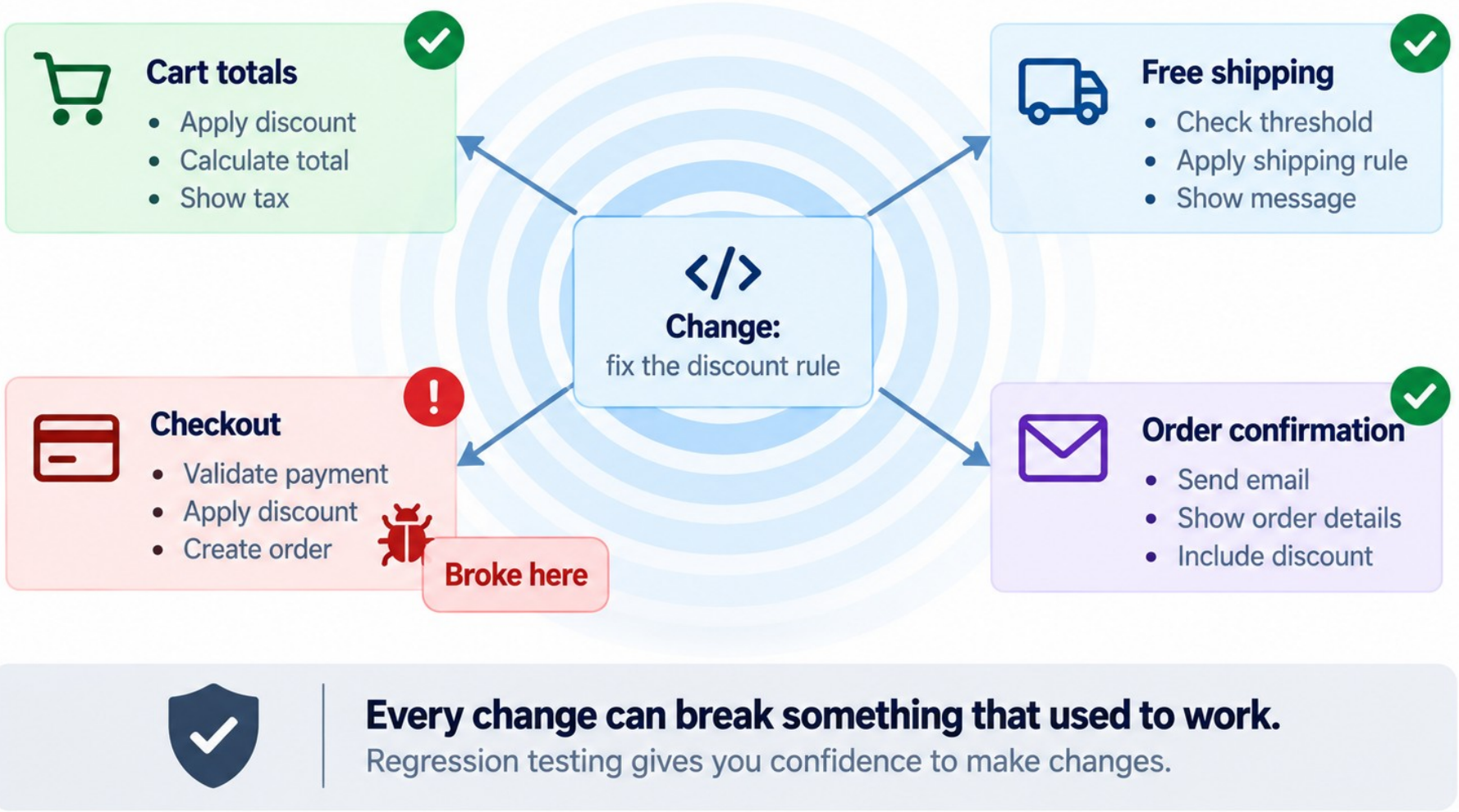
REGRESSION TESTING

A regression is something that **used to work** and doesn't anymore. Fixes and new features often break things nearby, so every change needs a check that the rest still works.

- The regression suite grows with every important test case and every fixed bug
- It runs before every release, and ideally on every change
- It's repetitive by nature, which makes it the first candidate for automation

© 2026 by Innovation In Software Corporation

REGRESSION TESTING

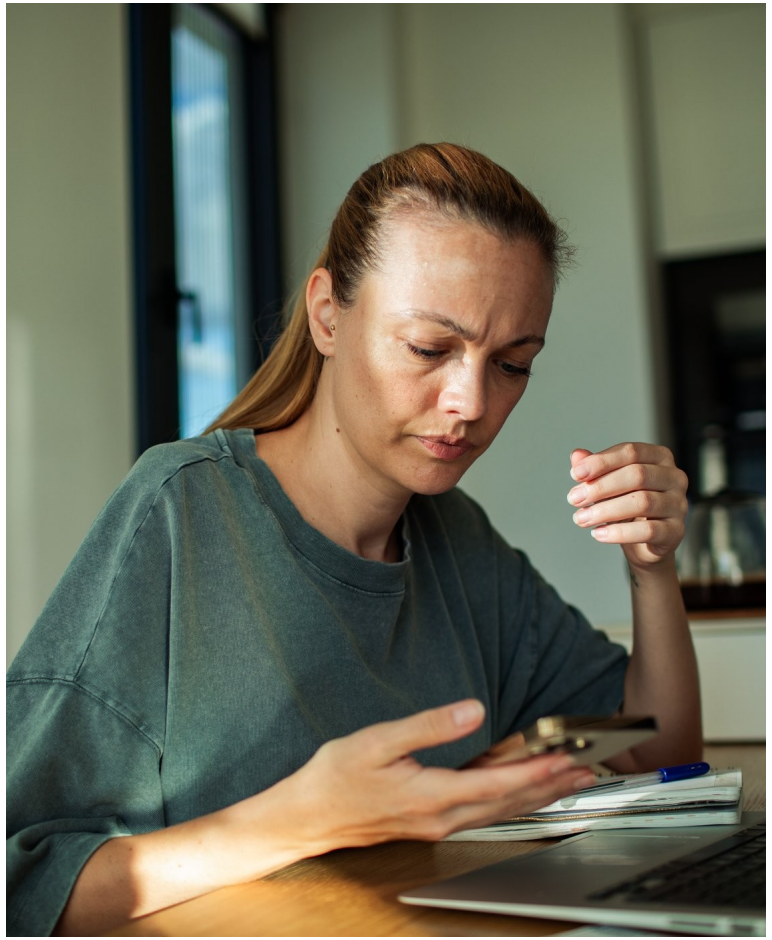


© 2026 by Innovation In Software Corporation

SMOKE VS. SANITY

The two are easy to confuse. **Smoke** asks, "Should we test this build at all?" It's quick and wide: does the catalog load, can someone log in, can a book go in the cart?

Sanity asks, "Is this one change good?" After a fix to the free-shipping rule, a sanity check tries an order right at the limit, one below it, and one above it, and nothing else.



© 2026 by Innovation In Software Corporation

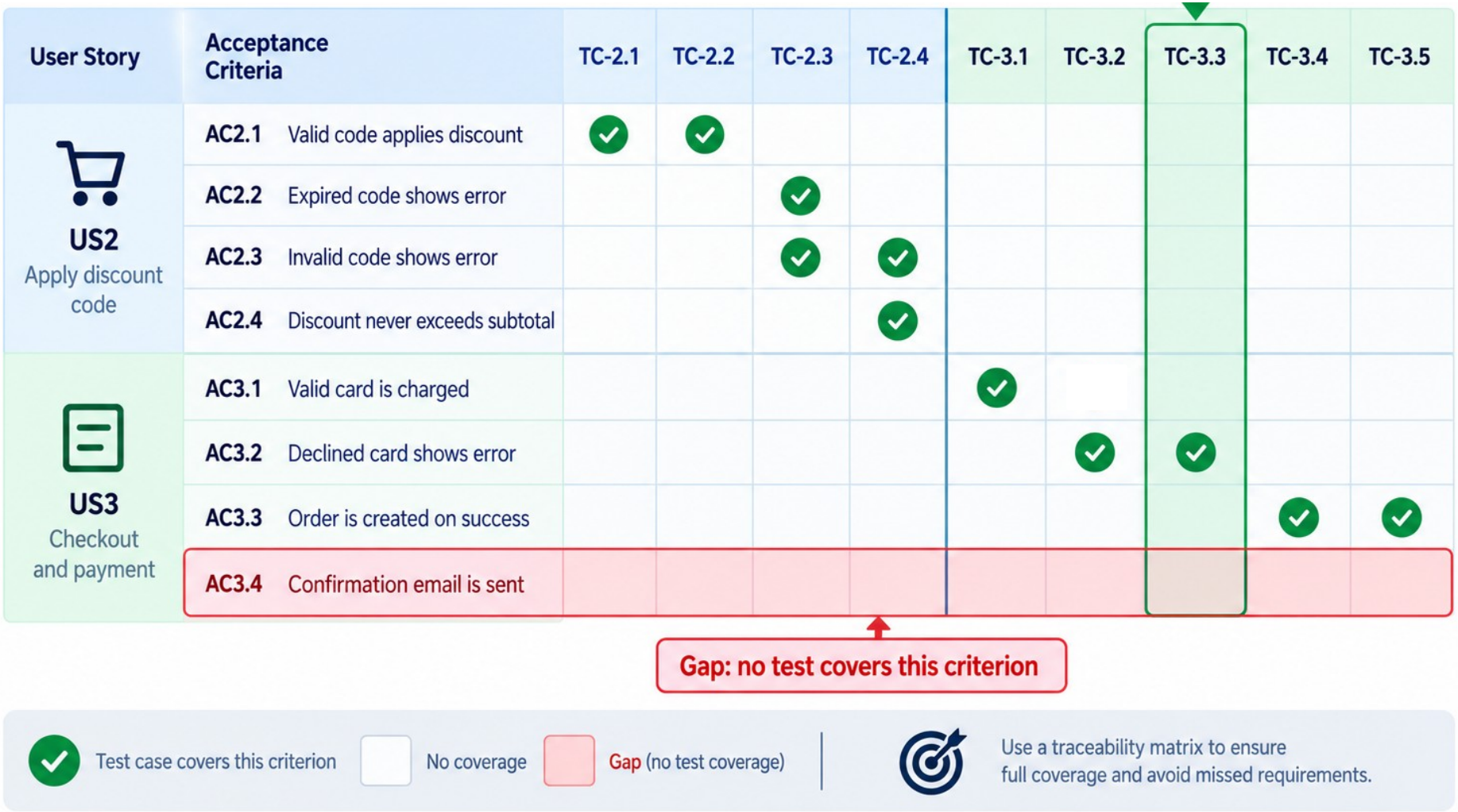
TRACEABILITY AND COVERAGE

A **traceability matrix** maps each acceptance criterion to the test cases that check it. Read it in one direction and you see what each test is for. Read it in the other and you see coverage.

Any acceptance criterion with no test case is a **gap**: behavior nobody will check until a user does.

© 2026 by Innovation In Software Corporation

TRACEABILITY MATRIX



© 2026 by Innovation In Software Corporation

KINDS OF COVERAGE

"How much have we tested?" has more than one answer.

Coverage	Measures	Example
Requirements	Acceptance criteria with at least one test	14 of 15 criteria covered
Risk	High-risk areas tested in depth	All P1 features have negative and edge cases
Code	Lines or branches run by tests	85% of discount code lines executed

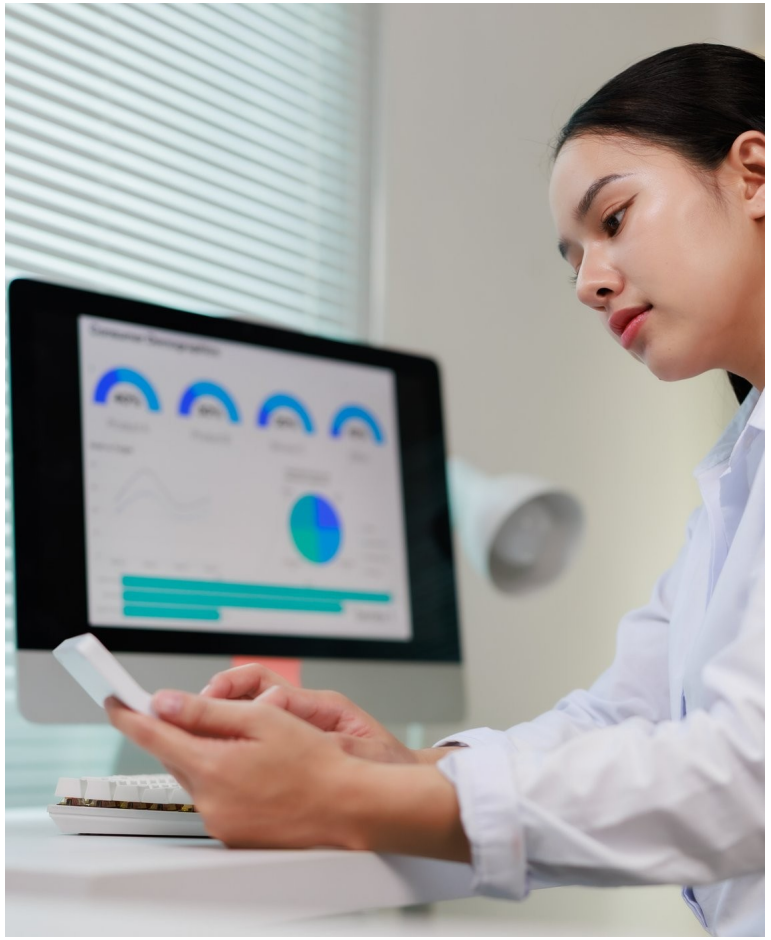
Requirements and risk coverage are QA's daily tools. Code coverage comes with automation (Day 2).

© 2026 by Innovation In Software Corporation

WHAT COVERAGE CAN'T TELL YOU

Coverage tells you what was **touched**, not whether it was checked **well**. A test that runs every line but never checks the result gives 100% coverage and catches nothing.

- 100% coverage doesn't mean bug-free
- Missing requirements have no coverage to measure
- Use coverage to find gaps, not to prove quality



© 2026 by Innovation In Software Corporation

THE STRANGER TEST

The real test of a test case is whether someone else can run it. Hand it to a teammate and watch, without helping.

- Did they get stuck on a step?
- Did they need data you didn't write down?
- Could they call the result pass or fail without asking you?

Every question they ask points to something missing from the test case.

© 2026 by Innovation In Software Corporation

LAB 1.3: STORIES TO TEST CASES

In this lab you turn BookNook's user stories into acceptance criteria and test cases the whole team can run.

- Write Given/When/Then acceptance criteria for the search and discount stories
- Write positive, negative, and edge test cases, tagged by level, type, suite, and priority
- Build a traceability matrix, find the gaps, and check the Definition of Done
- Swap test cases with a partner and run theirs exactly as written

© 2026 by Innovation In Software Corporation

POP QUIZ:

Which of these belongs in the team's Definition of Done, not in a story's acceptance criteria?

A. Given an expired code, the shopper sees "This code has expired."

B. Code is peer-reviewed and merged

C. Searching "prag" finds The Pragmatic Programmer

D. Results show 20 books per page





POP QUIZ:

Which of these belongs in the team's Definition of Done, not in a story's acceptance criteria?

A. Given an expired code, the shopper sees "This code has expired."

B. Code is peer-reviewed and merged

C. Searching "prag" finds The Pragmatic Programmer

D. Results show 20 books per page



POP QUIZ:

A developer just fixed the free-shipping rule. You check orders just below, at, and above the limit, and nothing else. What kind of testing is this?

A. Sanity testing

B. Smoke testing

C. Full regression testing

D. Acceptance testing



POP QUIZ:

A developer just fixed the free-shipping rule. You check orders just below, at, and above the limit, and nothing else. What kind of testing is this?

A. Sanity testing

B. Smoke testing

C. Full regression testing

D. Acceptance testing





POP QUIZ:

Your traceability matrix shows an acceptance criterion with no test cases. What does that mean?

A. The criterion is already done

B. There is a coverage gap

C. The test cases are too specific

D. The criterion belongs in the Definition of Done



POP QUIZ:

Your traceability matrix shows an acceptance criterion with no test cases. What does that mean?

A. The criterion is already done

B. There is a coverage gap

C. The test cases are too specific

D. The criterion belongs in the Definition of Done



DAY 1 WRAP-UP

Today you went from "What is quality?" to test cases a whole team can run.

- QA prevents defects; QC and testing find them
- Risk = Likelihood × Impact decides what to test first
- Scripted tests repeat what you know; exploration finds what you don't
- Bugs hide at boundaries, in invalid input, and where features interact
- Clear acceptance criteria and test cases make quality everyone's job

Tomorrow, the test cases you wrote today become automated tests.



© 2026 by Innovation In Software Corporation